NEXUSNEXUS

Comment ça marche — explication de chaque fonctionnalité

Document de présentation académique — Mai 2026

1. Véhicules et types
2. Routage sur réseau routier réel (A*)
3. Modèle de suivi de véhicule (Car-Following)
4. Feux de circulation
5. Système d'accidents
6. Gestion du trafic et horloge simulée
7. Zones géographiques (Geofences)
8. Rendu WebGL2 — Les véhicules
9. Trainées de mouvement (Trails)
10. Carte de congestion
11. Heatmap
12. Événements et incidents en direct
13. Communication WebSocket
14. Tableau de bord et contrôles
15. Import de carte .osm
16. Système de replay

■ 1. Les Véhicules — CAR · BUS · TRUCK · MOTORCYCLE · EMERGENCY

On a cinq types de véhicules dans la simulation, chacun avec un comportement et une apparence différents. C'est la base de tout — chaque véhicule est un objet Java qui vit dans le backend et dont l'état est diffusé en continu vers le frontend.

Quand la simulation démarre, le **VehicleGenerator** crée N véhicules (500 par défaut, jusqu'à 5 000) et les place chacun sur une arête aléatoire du graphe routier OSM. Chaque véhicule reçoit une vitesse de croisière qui dépend de son type — une moto va naturellement plus vite qu'un camion. Cette vitesse de croisière est la vitesse *désirée* quand la route est dégagée ; la vitesse *réelle* peut être beaucoup plus basse selon le trafic autour de lui.

Côté frontend, chaque type a une couleur fixe et une taille en pixels différente. Les voitures sont bleues, les bus verts, les camions orange, les motos violettes, et les véhicules d'urgence rouges — comme ça on reconnaît immédiatement qui est qui sur la carte, même avec des milliers de véhicules.

5 Types	**5 000** Max véhicules	**500** Défaut	**VehicleType.java** Fichier backend	**vehicle.model.ts** Fichier frontend

■ *Les véhicules EMERGENCY sont spéciaux : ils ne peuvent jamais être accidentés et sont immunisés contre les blocages du système d'accidents.*

■■ 2. Routage sur réseau routier réel (A*) —

OpenStreetMap + algorithme A*

Les véhicules ne bougent pas en ligne droite ou de façon aléatoire. Ils suivent de vraies routes extraites d'OpenStreetMap, calculées par l'algorithme A — le même algorithme que les GPS.*

Au démarrage, le backend télécharge le réseau routier de la zone configurée (Paris par défaut) via l'**API Overpass**. Ce réseau est transformé en un **graphe orienté** : chaque intersection OSM devient un nœud identifié par un ID unique, et chaque segment de rue entre deux intersections devient une arête. Ce graphe est *immuable* une fois chargé — personne ne le modifie pendant la simulation, donc on n'a pas besoin de le protéger par des verrous à chaque lecture.

Quand un véhicule a besoin d'une nouvelle destination, on appelle **AStarPathfinder.findPath()**. L'algorithme explore le graphe en partant du nœud courant, en utilisant la **distance Haversine** jusqu'à la destination comme heuristique — ça garantit qu'il ne surestime jamais le coût, donc le chemin trouvé est toujours optimal. Le résultat est une liste ordonnée d'IDs de nœuds : [départ, n1, n2, ..., arrivée]. Le véhicule suit cette liste segment par segment, et quand il arrive à destination, on lui calcule un nouveau chemin vers un nœud aléatoire. Ça crée un mouvement infini et naturel sur les routes.

1. Au démarrage : téléchargement du réseau OSM via Overpass, construction du graphe nœuds+arêtes
2. VehicleGenerator place chaque véhicule sur une arête aléatoire du graphe
3. Premier tick : A* calcule un chemin complet depuis le nœud courant vers un nœud aléatoire
4. À chaque tick : le véhicule avance sur son arête (progress 0.0 → 1.0)
5. Arrivée au nœud destination : on passe au segment suivant dans la liste du chemin A*
6. Fin du chemin : nouveau A* vers une nouvelle destination aléatoire — et ainsi de suite

Haversine	4 000	400	O((V+E) log V)
Heuristique	Max nœuds/calcul	Max A*/tick	Complexité

■ *On a mis une limite de 400 calculs A* par tick. Sans ça, si 500 véhicules arrivent à destination en même temps, le tick exploserait. Les véhicules en attente restent sur leur arête et obtiennent leur chemin au prochain tick.*

AStarPathfinder.java	RoadGraph.java	RoadNetworkLoader.java	SimulationEngine.java

■ 3. Modèle de suivi de véhicule (Car-Following)

— Files naturelles, réalisme du trafic

Les véhicules ne se traversent pas. Si un véhicule est trop proche de celui devant, il ralentit automatiquement. C'est ce qu'on appelle le car-following, et c'est ce qui crée des bouchons réalistes après un accident.

À chaque tick, on a besoin de savoir pour chaque véhicule combien il y a de place devant lui sur la même portion de route. La méthode **computeFrontGaps()** fait ça en triant tous les véhicules par *(arête, progression)*. Une fois triés, deux véhicules consécutifs sur la même arête orientée sont forcément un suiveur et un leader. Le gap entre eux est calculé directement : (progression du leader – progression du suiveur) × longueur de l'arête en mètres. Ce tri O(n log n) est la seule opération coûteuse, et on réutilise le même buffer à chaque tick pour ne jamais allouer de mémoire.

Une fois le gap connu, **advanceOnRoad()** calcule la vitesse cible du véhicule. Si la route est dégagée (gap ≥ 45m), il roule à sa vitesse de croisière. Si le gap descend entre 6m et 45m, la vitesse cible diminue proportionnellement. En dessous de 6m, arrêt complet. Et la vitesse réelle ne change pas d'un coup — elle est ajustée avec des bornes d'accélération et de décélération, ce qui donne des transitions douces et réalistes.

1. Tous les véhicules sont triés par (arête_de, arête_vers, progress) — un seul tri O(n log n)
2. Deux voisins sur la même arête → gap = différence de progression × longueur arête
3. Gap ≥ 45m → vitesse cible = vitesse de croisière (route libre)
4. Gap entre 6 et 45m → vitesse cible proportionnelle (ralentissement progressif)
5. Gap < 6m → vitesse cible = 0 (arrêt complet : voiture collée)
6. Accélération max : 9 km/h/s | Décélération max : 24 km/h/s (transitions fluides)

< 6 m	**6–45 m**	**≥ 45 m**	**9 km/h/s**	**24 km/h/s**
Arrêt complet	Ralentissement	Route libre	Accel max	Frein max

`SimulationEngine.java → computeFrontGaps()`	`SimulationEngine.java → advanceOnRoad()`

■ 4. Feux de circulation — Automatiques sur les vraies intersections

On a un système de feux de signalisation qui s'installe automatiquement sur les grandes intersections du graphe OSM. Les véhicules s'arrêtent aux feux rouges et les files se forment naturellement grâce au car-following.

Le **TrafficLightSystem** analyse le graphe routier juste après son chargement. Il parcourt tous les nœuds et sélectionne ceux qui ont au moins 4 voisins directs — ce sont de vraies grandes intersections, pas des virages ou des impasses. Il en sélectionne au maximum 350 pour garder le rendu et les calculs rapides. Chaque feu sélectionné reçoit un offset de cycle aléatoire, ce qui fait que tous les feux ne sont pas synchronisés — exactement comme dans la réalité.

Ce qui est intéressant dans notre implémentation, c'est qu'on ne stocke pas l'état du feu (rouge ou vert) dans une variable qu'on met à jour à chaque tick. Au lieu de ça, on le **calcule à la volée** avec une formule arithmétique simple : *phase = (maintenant + offset) modulo 16 000*. Si phase < 9 000 → vert. Sinon → rouge. Zéro mutation, zéro thread de fond pour les feux — juste du calcul pur.

Côté mouvement des véhicules : quand un véhicule s'approche d'un nœud qui a un feu et que ce feu est rouge, sa vitesse cible passe à zéro et sa progression est bloquée à 0.999 — il ne franchit jamais la ligne d'arrêt. Dès que le feu passe au vert, il réaccélère normalement.

1. Après chargement du graphe : sélection des nœuds avec ≥ 4 voisins (vraies intersections)
2. Maximum 350 feux, sélection déterministe (même graphe = mêmes feux)
3. Chaque feu reçoit un offset aléatoire pour désynchroniser les cycles
4. Chaque tick : phase = (now + offset) % 16 000 → vert si < 9 000ms, rouge sinon
5. Véhicule à < 50m d'un nœud avec feu rouge → vitesse cible = 0
6. Progress bloquée à 0.999 → le véhicule ne dépasse jamais le feu

9 s	7 s	16 s	350	à la volée
Phase verte	Phase rouge	Cycle	Max feux	Calcul état

`TrafficLightSystem.java`	`SimulationController.java` → `/api/simulation/lights`

■ 5. Système d'accidents — Bouchons émergents, obstacles statiques

Un accident dans notre simulation, c'est simplement un véhicule figé sur place pendant une durée aléatoire. On n'a pas eu besoin de coder un système d'embouteillage spécial — les bouchons apparaissent tout seuls grâce au car-following.

L'**AccidentSystem** est une classe utilitaire appelée à chaque tick. Elle maintient un timer : toutes les 10 à 26 secondes (intervalle aléatoire), elle choisit un véhicule au hasard dans la liste et le "wreck". Concrètement, ça veut dire qu'on met son champ wreckedUntilMs à une valeur dans le futur — entre 12 et 32 secondes plus tard. Pendant tout ce temps, **advanceOnRoad()** détecte ce champ, met la vitesse du véhicule à zéro, et ne le déplace plus. Il devient un obstacle statique sur la route.

Ce qui crée les bouchons, c'est le car-following. Le premier véhicule qui arrive derrière le véhicule accidenté voit son gap descendre sous 6m, donc il s'arrête. Le deuxième voit le premier s'arrêter, son gap descend aussi, et ainsi de suite. La file se propage naturellement vers l'arrière, exactement comme dans la réalité. Quand l'accident est terminé (wreckedUntilMs < maintenant), le véhicule reprend simplement sa route A*.

On a mis une limite de 5 accidents simultanés pour ne pas bloquer toute la carte. Les véhicules EMERGENCY ne peuvent jamais être accidentés — c'est une règle métier. La sévérité de l'accident dépend du type : camion et bus ont une sévérité de 3, voiture de 2, moto de 1. Cette sévérité est envoyée au frontend via l'API /api/simulation/events et détermine la taille du cercle rouge affiché sur la carte.

1. Toutes les 10 à 26s : l'AccidentSystem choisit un véhicule aléatoire non EMERGENCY
2. Il met wreckedUntilMs = maintenant + (12 à 32 secondes)
3. Pendant cet intervalle : speed = 0, le véhicule ne bouge plus
4. Les véhicules derrière détectent gap < 6m → ils s'arrêtent → file naturelle
5. L'incident est envoyé via GET /api/simulation/events → affiché sur la carte
6. Popup au clic : type, sévérité, compte à rebours jusqu'à la fin
7. Fin du timer : le véhicule reprend sa route A* normalement

5	12 s	32 s	immunisé	3 (camion)
Max simultanés	Durée min	Durée max	EMERGENCY	Sévérité max

AccidentSystem.java	IncidentUpdate.java	incident-layer.service.ts

■■ 6. Gestion du trafic et horloge simulée — 60×

temps réel, heures de pointe automatiques

On a une horloge simulée qui tourne 60 fois plus vite que le temps réel. Ça permet de traverser une journée entière en 24 minutes, de voir les heures de pointe se déclencher automatiquement, puis la nuit avec ses routes dégagées.

Le **TrafficTimeService** gère cette horloge. À chaque tick de 100ms réel, l'horloge avance de 6 minutes simulées (100ms × 60 = 6 000ms simulées = 6 minutes). L'heure est stockée comme un AtomicLong thread-safe, parce qu'elle est lue à la fois par le thread de simulation et par les threads HTTP. L'horloge démarre à 07h00 simulé — juste avant les heures de pointe du matin.

L'heure simulée détermine automatiquement le **mode de trafic** actif. En RUSH_HOUR (7h-9h et 17h-19h), le facteur de vitesse est de 0,55 : tous les véhicules roulent à seulement 55% de leur vitesse de croisière. Comme ils roulent plus lentement, les gaps réduisent, et les bouchons apparaissent naturellement autour des accidents et des feux. La nuit, le facteur monte à 1,35 — les routes sont dégagées. En plus du facteur de vitesse, chaque mode a une variance (jitter) appliquée aléatoirement à chaque véhicule, pour que les vitesses ne soient pas toutes identiques.

On peut aussi **forcer un mode manuellement** depuis le dashboard en cliquant sur le badge de mode ou en appuyant sur la touche R. Ça envoie un POST sur /api/simulation/traffic/override?mode=X. Pour revenir à l'auto, on fait le même appel sans paramètre.

1. Chaque tick : TrafficTimeService.advance(deltaMs) ajoute 6min simulées par 100ms réel
2. TrafficMode est déterminé par l'heure simulée (ou par l'override manuel)
3. dt_effectif = dt × speedMultiplier × mode.speedFactor × (1 ± jitter)
4. RUSH_HOUR → ×0,55 ±35% : routes lentes, files naturelles
5. NORMAL → ×1,0 ±15% : trafic standard
6. NIGHT → ×1,35 ±8% : routes dégagées et rapides

60×	**07h00**	**×0,55 ±35%**	**×1,0 ±15%**	**×1,35 ±8%**
Accélération	Départ	RUSH_HOUR	NORMAL	NIGHT

`TrafficTimeService.java`	`TrafficMode.java`	`SimulationController.java → /traffic`

■■ 7. Zones géographiques (Geofences) —

Smart-city, comptage temps réel

On a défini quatre zones géographiques sur la carte — des polygones qui représentent des scénarios smart-city typiques. Le backend compte en temps réel combien de véhicules simulés se trouvent dans chaque zone, et le frontend affiche ce comptage en direct.

Le **ZoneSystem** est un ensemble de polygones définis une fois au démarrage. On a quatre zones : une zone de faibles émissions (rectangle), une zone aéroport (cercle approximé en 24 segments), une zone de livraison au port (cercle en 20 segments), et une zone de sécurité scolaire (cercle en 16 segments). Ces quatre zones correspondent à des cas d'usage réels en smart-city : contrôle environnemental, gestion du trafic autour des aéroports, logistique urbaine, et sécurité aux abords des écoles.

Pour le comptage, on ne peut pas parcourir tous les véhicules et faire un test géométrique complet à chaque tick — ce serait trop lent. On a donc deux niveaux d'optimisation. D'abord, on ne recalcule que **4 fois par seconde** (pas à chaque tick de 100ms). Ensuite, chaque zone a une **bounding box** pré-calculée. Un véhicule hors de la bbox est rejeté en 4 comparaisons — la grande majorité des véhicules ne passeront jamais ce premier test. Pour les véhicules dans la bbox, on fait le test exact de **ray-casting** : on lance un rayon horizontal et on compte les intersections avec le polygone. Nombre pair = dehors, nombre impair = dedans.

1. Zones définies au démarrage : polygones GPS, bounding boxes pré-calculées
2. Chaque tick : update() vérifie si 250ms se sont écoulés depuis le dernier calcul
3. Si oui : parcourt tous les véhicules, test bbox d'abord (rejet rapide)
4. Si dans la bbox : test ray-casting exact O(nb_sommets)
5. Résultat : chaque zone a son comptage mis à jour
6. GET /api/simulation/zones → frontend → labels live sur la carte MapLibre

4	4 Hz (250ms)	Bounding box	Ray-casting
Zones	Refresh	Test rapide	Test exact

ZoneSystem.java	ZoneStatus.java	geofence-layer.service.ts

■ 8. Rendu WebGL2 — Les véhicules — GPU instancié, zéro DOM

C'est le cœur de la performance. Tous les véhicules — qu'ils soient 500 ou 5 000 — sont dessinés en un seul appel GPU. Zéro élément HTML, zéro SVG, zéro DOM. Tout passe par des buffers mémoire et des shaders GLSL.

On a implémenté un layer WebGL2 personnalisé pour MapLibre, le **VehicleRenderer**. Au démarrage, on alloue un buffer GPU de 20 000 slots — chaque slot représente un véhicule et contient 9 flottants : position (x, y en coordonnées Mercator), couleur RGB, taille, direction (sin et cos), et un flag "sélectionné". Ce buffer ne change jamais de taille — c'est un buffer de taille fixe qu'on remplit partiellement à chaque frame.

À chaque frame WebSocket reçue (10 fois/sec), on remplit les premiers N slots du buffer avec les données de chaque véhicule. Ensuite, bufferSubData() envoie uniquement les N premiers slots au GPU. Puis un seul appel drawArraysInstanced(count) demande au GPU de dessiner count instances du même quad — le shader fait le reste pour chaque instance.

Le shader vertex calcule une **taille constante en pixels** quelle que soit le niveau de zoom. La technique consiste à multiplier l'offset par .w (la composante homogène de la projection) avant de l'ajouter à la position, ce qui annule la division perspective que le GPU fait automatiquement. Le shader fragment dessine un cercle via la distance de Manhattan et smoothstep, avec un anneau lumineux pour le véhicule sélectionné.

Comme le serveur envoie 10 frames/sec mais le navigateur rend à 60fps, on fait de l'**interpolation côté client** : entre chaque frame serveur, on calcule une position interpolée selon le temps écoulé, ce qui donne un mouvement parfaitement fluide même à basse fréquence réseau.

1. Initialisation : allocation de 20 000 slots GPU (720 KB, DYNAMIC_DRAW)
2. Chaque frame WebSocket : remplissage des N premiers slots (9 flottants par véhicule)
3. bufferSubData() → envoi des données au GPU (seulement N slots, pas les 20 000)
4. drawArraysInstanced(N) → le GPU dessine N cercles en un seul appel
5. Shader vertex : taille pixel constante quelle que soit le zoom (trick .w)
6. Shader fragment : cercle via smoothstep, glow pour le véhicule sélectionné
7. Entre les frames : interpolation linéaire pour lisser le mouvement à 60fps

20 000	9	1	60 fps	10 fps
Slots GPU	Flottants/véhicule	Appels GPU/frame	Rendu	Updates serveur

vehicle-renderer.ts	spatial-grid.ts	vehicle-selector.ts

■ 9. Trainées de mouvement (Trails) — Ring buffer

WebGL, trajectoires visibles

Les trainées montrent la trajectoire récente de chaque véhicule — une ligne lumineuse qui s'estompe vers l'arrière. On active ou désactive ça avec la touche T ou le toggle dans le dashboard.

Le **TrailRenderer** est un deuxième layer WebGL2 personnalisé. Chaque véhicule a un **ring buffer** de ses dernières positions GPS. Un ring buffer, c'est un tableau circulaire de taille fixe : quand il est plein, on écrase la position la plus ancienne. On n'alloue jamais de nouvelle mémoire — la taille est fixe du début à la fin.

À chaque frame WebSocket, on ajoute la nouvelle position de chaque véhicule dans son ring buffer. Pour le rendu, on parcourt le buffer dans l'ordre du plus récent au plus ancien et on dessine des segments de ligne WebGL2. L'alpha de chaque segment diminue proportionnellement à son ancienneté — le point le plus récent est complètement opaque, le point le plus ancien est presque invisible. Ça donne l'effet de trainée lumineuse.

Comme le VehicleRenderer, le TrailRenderer a son propre **VAO** (Vertex Array Object). Ça veut dire que son état OpenGL (quels buffers, quels attributs) est encapsulé dans cet objet. MapLibre ne peut pas accidentellement corrompre notre état GL en changeant ses propres bindings entre deux frames.

1. Chaque véhicule a un ring buffer de positions (taille fixe, jamais réalloué)
2. Chaque frame WebSocket : nouvelle position ajoutée en tête du ring buffer
3. Si le buffer est plein : écrasement de la position la plus ancienne
4. Rendu : parcours du buffer du plus récent au plus ancien
5. Alpha de chaque segment = 1 - (index / taille_buffer) → effet d'estompage
6. Tout le buffer est envoyé au GPU en un seul bufferSubData()

trail-renderer.ts

■■ 10. Carte de congestion — Grille 40×30, WebGL2 instancié

La carte de congestion montre d'un coup d'œil où le trafic est fluide (vert) et où il est saturé (rouge). On l'active avec la touche C. C'est un renderer WebGL2 fait from scratch, indépendant du renderer de véhicules.

L'idée est de découper toute la zone de simulation en une **grille régulière** de 40 colonnes par 30 lignes, soit 1 200 cellules. Chaque cellule représente environ 350m × 300m de terrain. À l'initialisation du **CongestionRenderer**, on précalcule les coordonnées Mercator du centre de chaque cellule — ces valeurs ne changent jamais, donc on ne les recalcule pas à chaque frame.

À chaque mise à jour (environ 2 fois par seconde, pas à chaque frame), on parcourt tous les véhicules et on les assigne à une cellule selon leur position GPS. On accumule pour chaque cellule le nombre de véhicules et la somme de leurs vitesses. Ensuite on calcule un **score de congestion** : *(densité / densité_max)* × *(1 – vitesse_moy / 120)*. Ce score vaut 0 si la cellule est vide ou si les véhicules roulent vite, et 1 si la cellule est dense et les véhicules sont quasi à l'arrêt.

Le score est mis dans le buffer d'instances GPU (5 flottants par cellule : centre X, centre Y, demi-largeur, demi-hauteur, score). Un seul appel drawArraysInstanced(1 200) dessine toute la grille. Le shader fragment fait le dégradé de couleur : vert pour score bas, jaune pour score moyen, rouge pour score élevé. Les cellules avec score < 0.05 sont discardées (invisibles) pour ne pas afficher une grille partout.

1. Initialisation : grille 40×30, centres Mercator pré-calculés pour les 1 200 cellules
2. Toutes les ~500ms : parcours de tous les véhicules, assignation à leur cellule
3. Par cellule : comptage + somme des vitesses pour calculer la vitesse moyenne
4. Score = (densité / max_densité) × (1 – vitesse_moy / 120)
5. Score → buffer GPU (5 flottants/cellule)
6. drawArraysInstanced(1200) → shader GLSL fait le dégradé vert→jaune→rouge
7. Cellules vides : score < 0.05 → discard dans le fragment shader (transparentes)

1 200	40 × 30	~2 Hz	1	vert fluide	rouge saturé
Cellules	Grille	Refresh	Appels GPU	Score 0	Score 1

`congestion-renderer.ts`

■■ 11. Heatmap — Couche native MapLibre, densité GPS

La heatmap montre la densité de présence des véhicules sur la carte — pas leur congestion instantanée, mais les zones où il y a le plus de véhicules. On l'active avec la touche H.

Contrairement à la grille de congestion qu'on a codée from scratch, la heatmap utilise la **couche heatmap native de MapLibre GL JS**. MapLibre est capable de prendre une source GeoJSON de points et de calculer lui-même la densité et le rendu thermique — tout ça en GPU, dans ses propres shaders internes.

Le **HeatmapService** crée une source GeoJSON avec la liste de tous les véhicules convertis en features géographiques (point + coordonnées). À chaque frame WebSocket reçue, on appelle source.setData() avec la nouvelle liste de points. MapLibre prend en charge tout le reste : calcul du kernel de densité, interpolation de couleur, rendu GPU. On configure juste quelques paramètres comme le rayon de diffusion et l'intensité, et MapLibre fait les gradients vert → jaune → rouge automatiquement.

La couche est initialement masquée (visibility: none). Quand l'utilisateur appuie sur H, on appelle simplement map.setLayoutProperty('nexus-heatmap', 'visibility', 'visible'). Pour la désactiver, on remet à 'none'. Simple et efficace — MapLibre gère tout le rendu.

1. Initialisation : création d'une source GeoJSON vide + couche heatmap MapLibre
2. Chaque frame WebSocket : conversion des VehicleUpdate en features GeoJSON (points GPS)
3. source.setData(geojson) → MapLibre recalcule la densité et re-rend
4. MapLibre fait le dégradé de couleur et le rendu GPU en interne
5. Touche H ou toggle dashboard : setLayoutProperty visibility visible/none

heatmap.service.ts	map.component.ts

■ 12. Événements et incidents en direct — Polling

REST, cercles MapLibre

Les incidents affichés sur la carte ne sont pas des données fictives. Chaque point rouge correspond à un vrai véhicule accidenté dans la simulation à ce moment précis. On l'active avec la touche I.

Le **IncidentLayerService** interroge le backend toutes les **1,5 secondes** via GET /api/simulation/events. Le backend parcourt tous les véhicules et retourne ceux dont wreckedUntilMs > maintenant. Chaque incident est converti en feature GeoJSON avec ses propriétés : type, sévérité, position, label, et timestamp d'expiration.

Ces features sont injectées dans une source GeoJSON MapLibre, et une couche de type **circle** les affiche sur la carte. Le rayon du cercle est interpolé selon le zoom et la sévérité : à zoom 10, un accident de sévérité 3 a un cercle de 12px ; à zoom 14, il monte à 21px. La couleur est toujours rouge pour les accidents. Un contour blanc autour du cercle le rend bien visible sur tous les fonds de carte.

Quand on clique sur un incident, une **popup MapLibre** apparaît avec le type, le label (par exemple "TRUCK accident"), et un **compte à rebours en secondes** calculé à partir du expiresAtMs. Le curseur passe en mode pointer au survol pour indiquer que c'est cliquable.

1. Toutes les 1,5s : GET /api/simulation/events → liste des véhicules wrecked
2. Conversion en FeatureCollection GeoJSON (un Feature par incident)
3. source.setData() → MapLibre met à jour les cercles sur la carte
4. Rayon = zoom × sévérité interpolé (expression MapLibre)
5. Clic sur un circle → Popup avec label + compte à rebours
6. Touche I : setLayoutProperty visibility visible/none

`incident-layer.service.ts`	`SimulationEngine.java → eventsSnapshot()`	`IncidentUpdate.java`

■ 13. Communication WebSocket — STOMP, 10 fps, bypass Angular

Le WebSocket est le canal principal entre le backend et le frontend. Ce n'est pas du polling HTTP — le backend pousse les données proactivement 10 fois par seconde, et le frontend traite ça directement sans passer par Angular.

Côté backend, le **VehicleBroadcaster** est annoté @Scheduled avec une fréquence de 100ms. À chaque exécution, il appelle tick(deltaMs) sur le moteur de simulation, récupère un snapshot de tous les véhicules, et envoie la frame via SimpMessagingTemplate.convertAndSend() vers le topic /topic/vehicles. Tous les clients connectés reçoivent la frame simultanément.

La frame est un objet JSON avec trois champs : le timestamp de génération (pour mesurer la latence), le nombre de véhicules, et la liste des mises à jour. Chaque mise à jour contient l'ID, la position (lat/lon), la vitesse, la direction, le type, et le flag wrecked.

Côté frontend, le **WebSocketService** crée une connexion STOMP over SockJS (SockJS gère le fallback si WebSocket n'est pas disponible). Le point critique pour la performance : les données reçues sont passées **directement aux renderers WebGL**, sans passer par Angular. Angular a un système de détection de changements — si on passait les données par là, il vérifierait des milliers de bindings à chaque frame et l'app freezerait. En bypassant Angular, le WebSocket et le WebGL tournent complètement en dehors du cycle de détection de changements.

1. Backend : @Scheduled(100ms) → tick() → snapshot() → convertAndSend(/topic/vehicles)
2. WebSocket STOMP : frame JSON → tous les clients simultanément
3. Frontend : réception dans WebSocketService (STOMP over SockJS)
4. Observable → MapComponent.subscribe() (hors Angular Zone)
5. Directement vers VehicleRenderer, TrailRenderer, CongestionRenderer
6. Latence mesurée : Date.now() - frame.timestamp → affiché dans les stats

STOMP / SockJS Protocole	10 fps (100ms) Fréquence	/topic/vehicles Topic	Oui (hors Zone) Bypass Angular	Mesurée Latence
`VehicleBroadcaster.java`	`websocket.service.ts`		`map.component.ts`	

■ 14. Tableau de bord et contrôles — REST API, Signals Angular, raccourcis

Le dashboard est le panneau de contrôle de la simulation. Il permet de changer le nombre de véhicules, la vitesse, de mettre en pause, de voir les statistiques en temps réel, de filtrer par type de véhicule, et d'activer ou désactiver tous les overlays visuels.

Le **DashboardComponent** communique avec le backend uniquement via REST API — pas via WebSocket. Chaque action utilisateur (curseur, bouton) déclenche un appel HTTP POST ou GET. Par exemple, déplacer le curseur de véhicules à 2 000 envoie un POST /api/simulation/vehicles?count=2000. Le backend ajoute ou supprime des véhicules sans interrompre la simulation.

Les statistiques affichées — FPS, temps de rendu, nombre de véhicules, latence WebSocket — ne viennent pas du backend. Elles sont calculées **côté frontend** par les renderers WebGL et transmises au dashboard via le **MapStateService**. Ce service utilise des **Signals Angular** (le nouveau système de réactivité d'Angular) pour propager les stats sans créer de dépendances circulaires entre les composants.

Les raccourcis clavier sont interceptés dans le **MapComponent** (le composant qui contient la carte). Quand on appuie sur une touche, un Subject RxJS émet un événement — par exemple togglePause$ pour la touche P. Le DashboardComponent est abonné à ce Subject et fait l'appel HTTP correspondant. Ça évite de mettre HttpClient dans le MapComponent.

1. Curseur véhicules → POST /api/simulation/vehicles?count=N
2. Curseur vitesse → POST /api/simulation/speed?multiplier=N
3. Bouton Pause (ou touche P) → POST /api/simulation/pause ou /resume
4. Badge mode trafic (ou touche R) → POST /api/simulation/traffic/override?mode=X
5. Stats FPS/latence : calculées par VehicleRenderer → MapStateService → Dashboard
6. Touche H/T/C/B/I/G → SelectionService toggle → VehicleRenderer/TrailRenderer/etc.

11	10	15	Frontend local
Panneaux UI	Raccourcis clavier	Endpoints REST	Stats source
`dashboard.component.ts`	`map-state.service.ts`	`selection.service.ts`	`SimulationController.java`

■ 15. Import de carte .osm — Hot-swap, n'importe quelle ville

On peut charger n'importe quelle ville du monde sans redémarrer l'application. Il suffit d'exporter un fichier .osm depuis OpenStreetMap et de le glisser-déposer dans l'interface. Le graphe routier est remplacé à chaud en quelques secondes.

Côté frontend, le **MapImportComponent** propose une interface drag-and-drop ou un bouton parcourir pour sélectionner un fichier .osm ou .osm.xml. Le fichier est envoyé au backend via un POST multipart/form-data vers /api/maps/upload. Pendant le traitement, un indicateur de chargement est affiché.

Côté backend, le **MapImportService** valide d'abord le fichier : est-ce un fichier .osm ou .osm.xml ? Est-il sous 300 MB ? Si oui, il sauvegarde le fichier dans le répertoire data/maps/ et parse le XML OSM via le **LocalFileRoadGraphProvider**. Ce provider lit le fichier, extrait tous les nœuds OSM (intersections) et toutes les voies (ways) marquées comme routes, et construit le même type de graphe orienté qu'on aurait eu avec l'API Overpass.

Une fois le nouveau graphe construit, on appelle **replaceGraph()** sur le moteur de simulation. Cette méthode prend le verrou en écriture (pour bloquer le tick pendant l'opération), installe le nouveau graphe, réinitialise les feux de circulation et l'AccidentSystem, et régénère tous les véhicules sur les nouvelles arêtes. L'opération est atomique : le tick repart immédiatement après, sur la nouvelle ville. Le FrameRecorder est aussi remis à zéro pour ne pas mélanger l'historique de deux villes différentes.

1. Frontend : drag-and-drop ou sélection d'un fichier .osm/.osm.xml
2. POST multipart/form-data → /api/maps/upload (max 300 MB)
3. Backend : validation du type et de la taille
4. Sauvegarde dans data/maps/ + parsing XML OSM → nouveau RoadGraph
5. replaceGraph() : verrou écriture → swap graphe + rebuild feux + régénération véhicules
6. Réponse : mapName, nodeCount, edgeCount, bounds (bounding box GPS)
7. Frontend affiche les stats de la nouvelle carte chargée

.osm .osm.xml	300 MB	Non	Atomique	Aucune
Formats	Taille max	Redémarrage	Swap	Données gardées

MapImportService.java	LocalFileRoadGraphProvider.java	map-import.component.ts	MapController.java

■ 16. Système de replay — Enregistrement automatique, lecture ajustable

Chaque frame de simulation est automatiquement enregistrée sur disque. On peut ensuite rejouer n'importe quel moment de la simulation, en pause, en avance rapide, ou au ralenti. Les renderers WebGL ne font aucune différence entre du live et du replay.

Côté backend, le **FrameRecorder** est appelé par le VehicleBroadcaster juste avant chaque envoi WebSocket. Il sérialise la SimulationFrame (timestamp + positions de tous les véhicules) et l'écrit dans un fichier sur le disque. Les fichiers sont listés via GET /api/replay/list et streamés via GET /api/replay/play/{fichier}.

Côté frontend, le **ReplayService** est une **machine à états** avec cinq états possibles : idle (aucun replay), loading (chargement en cours), ready (prêt mais pas en lecture), playing (lecture en cours), paused (pause). Quand on est en mode playing, le service récupère les frames depuis le backend et les émet comme un Observable, exactement comme le ferait le WebSocket live. Le MapComponent est abonné à cet Observable et passe les données aux renderers — les renderers ne savent pas si c'est du live ou du replay.

La vitesse de replay est contrôlable : en 0,5×, on rejoue au ralenti (utile pour analyser un accident précis) ; en 4×, on parcourt rapidement un long historique. La barre de scrub permet de sauter à n'importe quel moment de l'enregistrement. La touche Espace lance ou met en pause le replay. Quand on arrête le replay, l'application revient automatiquement au flux WebSocket live.

1. Backend : à chaque tick, FrameRecorder.record(frame) sauvegarde sur disque
2. GET /api/replay/list → liste des fichiers disponibles
3. Utilisateur charge un replay → ReplayService passe en état "loading"
4. GET /api/replay/play/{fichier} → stream de frames → état "ready"
5. Lecture : frames émises via Observable au même rythme que le WebSocket live
6. MapComponent reçoit les frames → même pipeline que le mode live (VehicleRenderer etc.)
7. Stop replay → reprise du flux WebSocket live automatiquement

5	0,5× / 1× / 2× / 4×	Espace	Aucune (transparent)
États	Vitesses	Raccourci	Détection live/replay
`FrameRecorder.java`	`ReplayController.java`	`replay.service.ts`	`replay-bar.component.ts`